# Numpy : Python

### 1. Create a 3×3 Identity Matrix with Float Data Type

```
arr = np.eye(3, dtype=float)
print(arr)
```

Output:

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

---

### 2. Create a 1D Array with Random Values between 0 and 1

```
arr = np.random.rand(10)
print(arr)
```

---

### 3. Create a 2D Array with Random Integer Values

```
arr = np.random.randint(0, 100, size=(3, 4))
print(arr)
```

---

### 4. Creating an Array Using a Custom Function

```
arr = np.fromfunction(lambda i: i**2, (10,), dtype=int)
print(arr)
```

Output:

```
[ 0  1  4  9 16 25 36 49 64 81]
```

---

### 5. Reshaping a 1D Array into a 2D Array

```
arr = np.arange(12)
reshaped = arr.reshape(3, 4)

print(reshaped)
```

---

### 6. Creating a 3×3 Array of Ones

```
arr = np.ones((3, 3))
print(arr)
```

---

### 7. Common Items Between Two Arrays

```
a = np.array([1,2,3,2,3,4,3,4,5,6])
b = np.array([7,2,10,2,7,4,9,4,9,8])

common = np.intersect1d(a, b)
print(common)
```

Output:

```
[2 4]
```

---

### 8. Remove All Items Present in Array b from Array a

```
a = np.array([1,2,3,4,5])
b = np.array([5,6,7,8,9])

result = np.setdiff1d(a, b)
print(result)
```

Output:

```
[1 2 3 4]
```

---

### 9. Limit Printed Elements to Maximum 6

```
a = np.arange(15)

np.set_printoptions(threshold=6)
print(a)
```

Output:

```
[ 0  1  2 ... 12 13 14]
```

---

### 10. Drop NaN Values from a 1D Array

```
arr = np.array([1,2,3,np.nan,5,6,7,np.nan])

result = arr[~np.isnan(arr)]
print(result)
```

Output:

```
[1. 2. 3. 5. 6. 7.]
```

---

### 11. First 20 Natural Numbers and 4×5 Array

```
arr1 = np.arange(1, 21)

arr2 = np.arange(1, 21).reshape(4, 5)

print(arr1)
print(arr2)
```

---

### 12. Properties of a 3D Array and Change Data Type

```
arr = np.arange(24).reshape(2, 3, 4)

print("Shape:", arr.shape)
print("Size:", arr.size)
print("Dimensions:", arr.ndim)
print("Datatype:", arr.dtype)

arr = arr.astype(np.float64)

print("New Datatype:", arr.dtype)
```

---

### 13. Reshape and Flatten

```
arr = np.arange(12)

arr2d = arr.reshape(3, 4)

flat = arr2d.ravel()

print(np.array_equal(arr, flat))
```

Output:

```
True
```

---

### 14. Element-wise Operations

```
a = np.array([1,2,3])
b = np.array([4,5,6])

print("Add:", a+b)
print("Subtract:", a-b)
print("Multiply:", a*b)
print("Divide:", a/b)
```

Division by zero:

```
a = np.array([1,2,3])
b = np.array([1,0,2])

print(a/b)
```

Output:

```
[1. inf 1.5]
```

NumPy returns `inf` and raises a warning.

---

### 15. Broadcasting Example

```
a = np.array([[1],
              [2],
              [3]])

b = np.array([10,20,30])

result = a + b

print(result)
```

Output:

```
[[11 21 31]
 [12 22 32]
 [13 23 33]]
```

Broadcasting expands:

- `(3,1)` → `(3,3)`
- `(3,)` → `(1,3)`

---

### 16. Replace Elements Greater Than 5

```
arr = np.random.randint(0, 11, (4, 4))

mask = arr > 5

arr[mask] = 5

print(arr)
```

---

### 17. Indexing and Slicing

```
arr = np.random.randint(0, 100, (4, 4))

print("Second Row:")
print(arr[1])

print("Last Column:")
print(arr[:, -1])

print("Top Left 2x2:")
print(arr[:2, :2])
```

---

### 18. NumPy Applications

## EDA

Calculate Mean

```
sales = np.array([100,120,140,130,150])

print(np.mean(sales))
```

## AI

Feature Vector

```
features = np.array([1.2, 2.5, 3.8])
normalized = features / np.linalg.norm(features)
```

## ML

Linear Regression Prediction

```
X = np.array([1,2,3])
w = np.array([0.5])

y_pred = X * w
```

## DL

Matrix Multiplication in Neural Networks

```
X = np.random.rand(3,4)
W = np.random.rand(4,2)

output = X @ W
```

---

### 19. Eigenvalues and Eigenvectors

```
A = np.random.randint(1, 10, (4,4))

eigenvalues, eigenvectors = np.linalg.eig(A)

reconstructed = (
    eigenvectors
    @ np.diag(eigenvalues)
    @ np.linalg.inv(eigenvectors)
)

print(np.allclose(A, reconstructed))
```

---

### 20. 3×3×3 Array and Flatten

```
arr = np.arange(27)

arr3d = arr.reshape(3,3,3)

flat = arr3d.flatten()

print(np.array_equal(arr, flat))
```

Output:

```
True
```

---

### 21. Matrix Multiplication Using dot() and @

```
A = np.array([[1,2],[3,4]])
B = np.array([[5,6],[7,8]])

print(np.dot(A, B))
print(A @ B)
```

Performance:

```
import time

A = np.random.rand(1000,1000)
B = np.random.rand(1000,1000)

start = time.time()
np.dot(A,B)
print("dot:", time.time()-start)

start = time.time()
A @ B
print("@:", time.time()-start)
```

Results are identical.

---

### 22. Broadcasting with 3D and 2D Arrays

```
a = np.random.randint(1,10,(2,1,4))
b = np.random.randint(1,10,(4,1))

result = a + b.T

print(result.shape)
```

Using newaxis:

```
result2 = a + b.T[np.newaxis,:,:]

print(result2.shape)
```

---

### 23. Replace Values Less Than 0.5 by Their Squares

```
arr = np.random.rand(4,4)

mask = arr < 0.5

arr[mask] = arr[mask]**2

print(arr)
```

---

### 24. Sequential 5×5 Array Operations

```
arr = np.arange(1,26).reshape(5,5)

# Diagonal
print(np.diag(arr))

# Middle row zero
arr[2] = 0

print(arr)

# Vertical flip
print(np.flipud(arr))

# Horizontal flip
print(np.fliplr(arr))
```

---

### 25. 4D Array and Mean Along Axis

```
arr = np.random.randint(0,100,(2,3,4,5))

sub = arr[:,1,:,:]

mean_axis = np.mean(arr, axis=2)

print(sub.shape)
print(mean_axis.shape)
```

---

### 26. Reshape (10,20) Array

```
arr = np.arange(200).reshape(10,20)

a = arr.reshape(20,10)

b = arr.reshape(5,40)

print(a.shape)
print(b.shape)
```

Observations:

- Original shape = `(10,20)`
- Reshaped = `(20,10)` and `(5,40)`
- Size remains `200`
- Dimensions remain `2`

---

### 27. Large Array with reshape() and ravel()

```
arr = np.arange(10000).reshape(100,100)

reshaped = arr.reshape(200,50)

flat = reshaped.ravel()

print(flat.shape)
```

`reshape()` changes structure without changing data.

`ravel()` converts to 1D view when possible.

---

### 28. Upper Triangular Matrix and Zero Lower Triangle

```
A = np.random.randint(1,20,(6,6))

upper = np.triu(A)
```

```
print("Upper Triangular:")
print(upper)

verify = upper.copy()

print("Lower Triangle Zero:")
print(verify)
```

Alternative:

```
A[np.tril_indices(6, -1)] = 0

print(A)
```

This sets all elements below the main diagonal to zero.